

Project Title

Sam

Charlie

Anusha

Mukund

Project Goal and Scope

The primary objective is to develop a photo-realistic 3D graphics rendering system in C++ utilizing advanced Path Tracing Techniques.

The specific target for the rendering competition is to accurately recreate an image of a chess scene [1]. Achieving this level of visual fidelity requires a synthesis of custom geometric modeling, designing material properties, and a Path Tracer that handles shadows, and glass surfaces.

Accomplishments

Bidirectional Path Tracing

We upgraded the core rendering engine to utilize Bidirectional Path Tracing [2]. In traditional Monte Carlo path tracing, rays are exclusively cast from the camera into the scene. Our implementation improves upon this by simultaneously generating sub-paths from both the camera (eye paths) and the emissive light sources (light paths). The length of both sub-paths is determined stochastically, utilizing Russian Roulette for unbiased termination.

Custom Scene Reconstruction and Material Formulation

To accurately replicate the reference image [1], the chess pieces and environment were modeled entirely from scratch rather than relying on pre-existing assets. The scene was meticulously arranged to match the exact framing and composition of the original photograph. Furthermore, to capture the distinct reflective properties of the various surfaces—such as the highly reflective floor tiles and the differing varnishes on the light and dark chess pieces—a novel hybrid material model, `DIFF_MIRROR`, was formulated. This custom material successfully bridges the gap between perfectly specular and purely diffuse surfaces to render complex specular highlights.

Dynamic Scene Configuration via Command-Line Interface

To optimize rendering efficiency and workflow flexibility, the system was extended to support dynamic scene configuration at runtime. Instead of relying on static, hardcoded parameters that require recompilation for every adjustment, critical rendering variables—such as camera position, image resolution, samples per pixel (SPP), and field of view (FOV)—can now be specified directly via command-line arguments. This enhancement significantly streamlined the rendering process, eliminating code clutter and allowing for the rapid, iterative generation of multiple required viewpoints from a single compiled executable.

Technical Details

Bidirectional Path Tracing

In standard unidirectional Path Tracing, the rendering equation is integrated exclusively by gathering light. Rays are cast from the camera into the scene, scattering at surfaces according to the material's BRDF until they randomly intersect an emissive light source or terminate via Russian Roulette. The final color at a surface is evaluated by accumulating the emitted radiance from the final hit point, attenuated by the material reflectance and geometric terms along the reverse path. However, this Monte Carlo integration method often struggles with complex indirect lighting, leading to high variance (noise) because camera paths rarely hit the light source purely by chance [2].

To evaluate the final pixel color using Bidirectional path tracing the integrator attempts to connect the vertices of the camera sub-path with the vertices of the light sub-path. For each attempted connection, a deterministic visibility check (shadow ray) is cast between the two endpoints. If an object occludes the line of sight between the vertices, no color contribution is added for that specific connection. If the path is unoccluded, the radiance is evaluated and accumulated similarly to standard Monte Carlo integration.

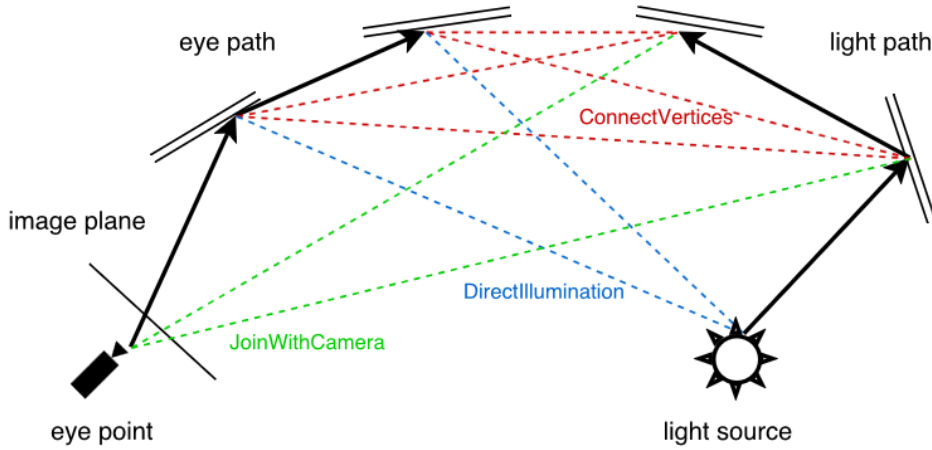


Figure 1: An illustration of Bidirectional Path Tracing. Sub-paths are generated simultaneously from both the camera (eye paths) and the emissive light sources (light paths), which are then connected to evaluate the final radiance.

The rendering equation used to describe traditional path tracing

$$L_o(p, \omega_o) = L_e(p, \omega_o) + \int_{\Omega} f_r(p, \omega_i, \omega_o) L_i(p, \omega_i) (\omega_i \cdot n) d\omega_i \quad (1)$$

The previous equation is evaluated in Recursively. A ray can be traced from the camera eye into the nearest hit point x , then sample a new direction ω_i above x and repeat these steps until some kind of termination condition, such as Russian roulette,

The rendering equation (1) can be transformed into a path integral formulation where instead of integrating over all directions of incoming light over the point p , we integrate over points on the surfaces of objects.

$$L_p = \int_D f_j(\vec{p}) dD \quad (2)$$

where L_p is the radiance that flows through a pixel, D is all possible light paths and $\vec{p}$ is all possible light paths.

This can be approximated using monte-carlo approximation as follows

$$L_p = \frac{1}{N} \sum_{i=0}^N \frac{f_j(\vec{p}_i)}{p(\vec{p}_i)} \quad (3)$$

where $p(\vec{p}_i)$ is the pdf of sampling path $\vec{p}_i$. The conversion into a pdf with respect to surface area $p(x)$ is written as:

$$p(x) = p(\omega) \left(\frac{N_{x'} \cdot x'x''}{\|x'' - x'\|} \right) \quad (4)$$

where $p(\omega)$ is the solid angle. Moreover, We know that

$$L_r(x, \omega_0) = L_{direct} + L_{indirect} \quad (5)$$

(5) can be written with conversion to surface area as

$$L_p = \frac{1}{N} \sum_{i=0}^N L_{p_i} \quad (6)$$

where L_{p_i} is the radiance of a single sample which is calculated by

$$L_{p_i} = \sum_{t=0}^{N_E} C_t \quad (7)$$

Where C_t is the contribution of a single path of length t and N_E is the maximum path length. Applying monte carlo integration on the contribution gives us

$$C_t = \frac{L_e(y_0, \overrightarrow{y_0x_t})}{p(y_0)} f_r(x_t, \overrightarrow{x_t y_0}, \overrightarrow{x_t x_{t-1}}) G(x_t \leftrightarrow y_0) \prod_{i=1}^{t-1} \frac{f_r(x_i, \overrightarrow{x_i x_{i+1}}, \overrightarrow{x_i x_{i-1}}) |N_{x_i} \cdot \overrightarrow{x_i x_{i+1}}|}{p(\overrightarrow{x_i x_{i+1}})} \quad (8)$$

The first part of the equation corresponds to the direct lighting from y_0 onto x_t , where $L_e(y_0, \overrightarrow{y_0x_t})$, is the amount of radiance from point y_0 going in the direction towards x_t . Then it is multiplied with the geometric term, and also with the BRDF. The rest of the equation belongs to indirect lighting. It is computed for each point on the path as the product of BRDF and the dot product, divided by the PDF relative to BRDF.

The geometric coupling term can be defined as

$$G(x \leftrightarrow x') = V(x \leftrightarrow x') \frac{|N_x \cdot \overrightarrow{x'x}| |N_{x'} \cdot \overrightarrow{xx'}|}{\|x - x'\|^2} \quad (9)$$

where $V(x \leftrightarrow x')$ is a visibility function, which is equal to 1 if a ray can be shot directly from point x to x' without hitting anything else, otherwise it is 0

The light tracing which represents a light path can be written using the path integral framework (2) as follows

$$L_p = \sum_{s=0}^{N_L} C_s \frac{\|P_{pixel} - P_{eye}\|^2}{(\cos \theta)^2 A_{pixel}} \quad (10)$$

where the sum represents path, from point y_0 to the maximum path length N_L , C_s is the single path contribution and the rest of the equation is the conversion from flux to radiance. Contribution C_s can be computed as

$$C_s = \frac{L_e(y_0, \overrightarrow{y_0y_1})}{p(y_0, \overrightarrow{y_0y_1})} f_r(y_s, \overrightarrow{y_s y_{s+1}}) G(y_s \leftrightarrow x_0) \frac{W_e(x_0, \overrightarrow{y_t x'_0})}{p(x_0)} \left(\prod_{i=1}^{s-1} \frac{f_r(y_i, \overrightarrow{y_i y_{i+1}}, \overrightarrow{y_i y_{i-1}}) |N_{y_i} \cdot \overrightarrow{y_i y_{i+1}}|}{p(\overrightarrow{y_i y_{i+1}})} \right) \quad (11)$$

The first part is the emitted light $L_e(y_0, \overrightarrow{y_0 y_1})$ from the light source at point y_0 . The PDF $p(y_0, \overrightarrow{y_0 y_1})$ is probability of selecting a ray in the direction $\overrightarrow{y_0 y_1}$. Considering a light that has uniform distribution of the emitted light, then probability density function is:

$$p(y_0, \overrightarrow{y_0 y_1}) = \frac{1}{A_{light} 2\pi} \quad (12)$$

y_0 can be sampled over the light area A_{light} in all directions over the hemisphere above the point y_0 . $W_e(x_0, \overrightarrow{y_t x_0})$ is the potential of the light ray going straight from the light source to the pixel, which is equal to 1, when the ray is in front of the camera, and when using a pinhole camera $p(x_0)$ is equal to 1 as well, because only x_0 can be chosen as a point.

The next part $f_r(y_s, \overrightarrow{y_s y_{s+1}}, \overrightarrow{y_s y_{s+1}}) G(y_s \leftrightarrow x_0)$ represents a direct ray from y_s to the eye x_0 . The third part of the equation is the product of: the BRDF, multiplied with the dot product between normal and direction $\overrightarrow{y_i y_{i+1}}$ and normalized by the PDF.

We have defined both the strategies of ray evaluation, and now Bidirectional path tracing can be defined as

$$L_p = \sum_{s=0}^{N_L} \sum_{t=0}^{N_E} w_{s,t} \cdot C_{s,t} \quad (13)$$

where $C_{s,t}$ is the unweighted contribution of a path with s vertices on the light path and t vertices on the eye path. A visibility check between s^{th} point on the eye path and t^{th} point on the light path is a part of the geometric term in contribution function. And finally, $w_{s,t}$ is the weight function.

As it can be seen, evaluation of the $C_{s,t}$ is independent on the eye and light path; therefore, there are four important cases:

- $s = 0, t = 0$ light is directly visible from the eye, the evaluation can be done with direct path between eye the point x_0 and y_0
- $s > 0, t = 0$ in this case use light tracing defined in (11)
- $s = 0, t > 0$ corresponds to the classic path tracing case defined in (8)
- $s > 0, t > 0$ the final case is the evaluation of radiance from the eye path t connected with s vertices of the light path, reaching the pixel, defined as follows

$$\begin{aligned} C_{s,t} = & \frac{L_e(y_0, \overrightarrow{y_0 y_1})}{p(y_0, \overrightarrow{y_0 y_1})} G(y_s \leftrightarrow x_t) f_r(x_t, \overrightarrow{x_t y_s}, \overrightarrow{x_t x_{t-1}}) f_r(y_s, \overrightarrow{y_s y_{s+1}}, \overrightarrow{y_s x_t}) \\ & \times \left(\prod_{i=1}^{t-1} \frac{f_r(x_i, \overrightarrow{x_i x_{i+1}}, \overrightarrow{x_i x_{i-1}}) |N_{x_i} \cdot \overrightarrow{x_i x_{i+1}}|}{p(\overrightarrow{x_i x_{i+1}})} \right) \\ & \times \left(\prod_{i=1}^{s-1} \frac{f_r(y_i, \overrightarrow{y_i y_{i+1}}, \overrightarrow{y_i y_{i-1}}) |N_{y_i} \cdot \overrightarrow{y_i y_{i+1}}|}{p(\overrightarrow{y_i y_{i+1}})} \right) \end{aligned} \quad (14)$$

The most effective way to use multiple importance sampling in order to create a weight function is with the power heuristic:

$$w_{s,t} = \frac{p_s^\beta}{\sum_{i=0}^{s+t-1} p_i^\beta} \quad (15)$$

where the recommended value for $\beta = 2$

where p_i is defined as the density for generating path $x_{s,t}$ using i sub light path vertices and $s+t-i$ eye sub path vertices :

$$p_i = p_{i,s+t-i}(x_{s,t}^\rightarrow) \quad (16)$$

The algorithm suggested in [2] was implemented and is as follows:

Algorithm 1 BDPT

```

function CASTRAYBIDIRECTIONAL(scene) define number of light and eye paths using Russian
Roulette
  for all lightRays and cameraRays do
     $inter \leftarrow intersect(ray)$ 
     $w_-, p \leftarrow sample(inter)$ 
     $f = evaluateColor(inter, w_-)$ 
     $vertex.inter \leftarrow inter$ 
     $vertex.beta \leftarrow \max(0, inter.normal \cdot w_-) \times f \times vertex_{previous}.beta / p / RR$ 
     $ray \leftarrow ray(inter, w_-)$ 
  end for
  for all Vertices do
     $p_{s,s+1} \leftarrow \frac{p_{s-1,s} \times (w_{-s+1} \cdot w_{-s}) \times p_{vertex}}{(\|w_{-s} - w_{-s-1}\|)^2}$ 
  end for
   $pSum \leftarrow \sum_{s,t} (p_{camera\_s} \times p_{light\_t})^2$ 
   $hitColor \leftarrow 0$ 
  for all Camera Vertices do
    for all Light Vertices do
      if Nothing blocking between verticies then
         $w_- \leftarrow lightVertex.coords - cameraVertex.coords$ 
         $\theta \leftarrow \max(0, inter.normal \cdot w_-)$ 
         $connectingBeta \leftarrow \frac{evaluateColor(cameraVertex, w_-) \times \theta}{p_{cameraVertex} \times RR}$ 
         $G \leftarrow \frac{|N_{cameraVertex} \overrightarrow{lightVertex.coords - cameraVertex.coords}| \cdot N_{lightVertex.coords} \cdot cameraVertex.coords \cdot lightVertex.coords}{\|lightVertex.coords - cameraVertex.coords\|^2}$ 
         $color \leftarrow cameraVertex.beta \times lightVertex.beta \times connectingBeta \times G$ 
         $hitColor \leftarrow hitColor + \frac{color \times (p_{cameraVertex} \times p_{lightVertex})^2}{pSum}$ 
      end if
    end for
  end for
  return  $hitColor$ 
end function

```

Note that semi-reflectivity was handled by giving each ray percentage chance to bounce off of a material based on that material's index of refraction and the angle of incidence. If the ray did not bounce, the material was handled as though it was diffuse. Otherwise, the ray was reflected, and the reflective surface is not added to the list of vertices.

The implementation in C++ is as follows

```
Vector3f Scene::castRayBidirectional(const Ray &ray, int depth, bool shadows_on)
const {
    Vector3f hitColor = Vector3f(0);
    auto inter = intersect(ray); // find the closest intersection of the ray and the
    objects
    while (depth <= maxDepth && inter.happened && inter.material->m_type ==
        DIFF_MIRROR && fresnel(ray.direction.normalized(), inter.normal.normalized(),
        inter.material->ior) < get_random_float()) {
        inter = intersect(Ray(inter.coords + inter.normal.normalized() * EPSILON,
        reflect(ray.direction.normalized(), inter.normal.normalized())));
        depth++;
    }
    if (!inter.happened) return backgroundColor; // if no intersection, return
    background color
    if (inter.material->hasEmission()) return inter.material->getEmission();

    int numRaysFromLight = 1;
    while (get_random_float() <= RussianRoulette) {
        numRaysFromLight++;
    }

    int numRaysFromCamera = 1;
    while (get_random_float() <= RussianRoulette) {
        numRaysFromCamera++;
    }

    // Determine start of light's path
    Intersection lightInter;
    float lightPdf;
    sampleLight(lightInter, lightPdf);

    Vert ints[numRaysFromCamera + numRaysFromLight]; // Intersections/beta values of
    both the path from the camera and the path from the light

    Vector3f w_s[numRaysFromCamera + numRaysFromLight]; // Incoming w_ value for
    each intersection
    int totalRaysFromCamera = numRaysFromCamera;
    ints[0].inter = inter;
    w_s[0] = ray.direction;
    ints[totalRaysFromCamera].inter = lightInter;

    Vector2f st; // texture coordinates (u, v)
    Vector3f N; // normal

    // Start path from camera
    Vert start;
    start.inter = inter;
    start.beta = Vector3f(1);
    ints[0] = start;

    // Start path from light (light path starts with light at index
    numRaysFromCamera and increases from there)
    Vert end;
    end.inter = lightInter;
    ints[numRaysFromCamera] = end;

    // Create path from camera
    for (int i = 0; i < numRaysFromCamera; i++) {
        N = inter.normal.normalized();
```

```

    st = inter.tcoords;

    // outgoing w_
    Vector3f w_ = inter.material->sample(w_s[i], N).normalized();
    float pdf = inter.material->pdf(w_s[i], w_, N);

    // calculate color
    Vector3f albedo = inter.obj->evalDiffuseColor(st);
    Vector3f f = inter.material->eval(w_, N, albedo);
    float cosTheta_ = std::max(0.0f, dotProduct(N, w_));

    ints[i].beta = ints[std::max(0, i - 1)].beta * f * cosTheta_ / pdf /
        RussianRoulette;
    // ints[i].inter = inter;

    if (i < numRaysFromCamera - 1) {
        // Calculate next intersection
        w_s[i + 1] = w_;
        ints[i + 1].inter = inter;
        inter = intersect(Ray(inter.coords + N*EPSILON, w_));
        while (depth <= maxDepth && inter.happened && inter.material->m_type ==
            DIFF_MIRROR && fresnel(w_, inter.normal.normalized(), inter.material
            ->ior) < get_random_float()) {
            inter = intersect(Ray(inter.coords + inter.normal.normalized() *
                EPSILON, reflect(w_, inter.normal.normalized())));
            depth++;
        }
        if (!inter.happened) {
            numRaysFromCamera = i + 1;
        }
    }
}

// Calculate color of light
N = lightInter.normal.normalized();
w_s[totalRaysFromCamera] = lightInter.material->sample(-N, N).normalized();
float pdf = lightInter.material->pdf(-N, w_s[totalRaysFromCamera], N);
Vector3f albedo = lightInter.obj->evalDiffuseColor(lightInter.tcoords);
Vector3f f = lightInter.material->eval(w_s[totalRaysFromCamera], N, albedo);
float cosTheta_ = std::max(0.0f, dotProduct(N, w_s[totalRaysFromCamera]));
float r = std::pow(55, 2);
ints[totalRaysFromCamera].beta = lightInter.material->getEmission() / (pdf * r /
    std::abs(dotProduct(lightInter.normal, w_s[totalRaysFromCamera])));

// Create first path from light
if (numRaysFromLight > 1) {
    ints[totalRaysFromCamera + 1].inter = intersect(Ray(lightInter.coords + N*
        EPSILON, w_s[totalRaysFromCamera]));
    while (depth <= maxDepth && ints[totalRaysFromCamera + 1].inter.happened &&
        ints[totalRaysFromCamera + 1].inter.material->m_type == DIFF_MIRROR &&
        fresnel(w_s[totalRaysFromCamera], ints[totalRaysFromCamera + 1].inter.
        normal, ints[totalRaysFromCamera + 1].inter.material->ior) <
        get_random_float()) {
        ints[totalRaysFromCamera + 1].inter = intersect(Ray(ints[
            totalRaysFromCamera + 1].inter.coords + ints[totalRaysFromCamera +
            1].inter.normal.normalized() * EPSILON, reflect(w_s[
            totalRaysFromCamera], ints[totalRaysFromCamera + 1].inter.normal.
            normalized())));
        depth++;
    }
    if (!ints[totalRaysFromCamera + 1].inter.happened) {
        numRaysFromLight = 1;
    }
    inter = ints[totalRaysFromCamera + 1].inter;
}
}

```

```

// Calculate rest of path from light
for (int i = 1; i < numRaysFromLight; i++) {
    N = inter.normal.normalized();
    st = inter.tcoords;

    // outgoing w_
    Vector3f w_ = inter.material->sample(w_s[totalRaysFromCamera + i - 1], N).
        normalized();

    pdf = inter.material->pdf(w_s[totalRaysFromCamera + i - 1], w_, N);

    // Fix for weird error where intersection had no object associated with it
    if (inter.obj != nullptr){
        albedo = inter.obj->evalDiffuseColor(st);
        f = inter.material->eval(w_,N,albedo);
    }
    else {
        f = Vector3f(0.156,0.25,0.268);
    }

    // calculate color
    cosTheta_ = std::max(0.0f,dotProduct(N,w_));
    ints[totalRaysFromCamera + i].beta = ints[totalRaysFromCamera + std::max(0,
        i - 1)].beta * f * cosTheta_ / pdf / RussianRoulette;

    w_s[totalRaysFromCamera + i] = w_;
    ints[totalRaysFromCamera + i].inter = inter;

    // calculate next intersection
    inter = intersect(Ray(inter.coords + N*EPSILON, w_));
    while (depth <= maxDepth && inter.happened && inter.material->m_type ==
        DIFF_MIRROR && fresnel(w_, inter.normal.normalized(), inter.material->ior
    ) < get_random_float()) {
        inter = intersect(Ray(inter.coords + inter.normal.normalized() * EPSILON
            , reflect(w_, inter.normal.normalized())));
        depth++;
    }
    if (!inter.happened) {
        numRaysFromLight = i + 1;
    }
}

// Calculate probabilities for MIS
Intersection dummy;
float camera_ps[numRaysFromCamera];
float light_ps[numRaysFromLight];
ints[0].inter.obj->Sample(dummy, pdf);
camera_ps[0] = pdf;

for (int i = 1; i < numRaysFromCamera; i++) {
    camera_ps[i] = camera_ps[i-1] * dotProduct(w_s[i], w_s[i-1]) / std::powf(
        getLen(w_s[i], w_s[i-1]), 2) * ints[i].inter.material->pdf(w_s[i-1], w_s[
        i], ints[i].inter.normal);
}

light_ps[0] = lightPdf;
for (int i = 1; i < numRaysFromLight; i++) {
    light_ps[i] = light_ps[i - 1] * dotProduct(w_s[totalRaysFromCamera + i], w_s
        [totalRaysFromCamera + i - 1]) /
        std::powf(getLen(w_s[totalRaysFromCamera + i], w_s[totalRaysFromCamera +
            i - 1]), 2) * ints[totalRaysFromCamera + i].inter.material->pdf(w_s[
            totalRaysFromCamera + i-1], w_s[totalRaysFromCamera + i], ints[
            totalRaysFromCamera + i].inter.normal);
}

```



```

// Calculate sum of squares
float pSum = 0;

for (int i = 0; i < numRaysFromCamera; i++) {
    for (int j = 0; j < numRaysFromLight; j++) {
        pSum += std::powf(camera_ps[i] * light_ps[j], 2);
    }
}

// Calculate final color
for (int i = 0; i < numRaysFromCamera; i++) {
    for (int j = 0; j < numRaysFromLight; j++) {
        Vector3f start = ints[i].inter.coords + ints[i].inter.normal.normalized
            () * EPSILON;
        Vector3f current_w_ = (ints[totalRaysFromCamera + j].inter.coords -
            start).normalized();
        Intersection testOccluded = intersect(Ray(start, current_w_));
        Intersection oppositeTest = intersect(Ray(ints[totalRaysFromCamera + j].
            inter.coords + ints[totalRaysFromCamera + j].inter.normal * EPSILON,
            -current_w_));
        if (!shadows_on || testOccluded.happened && oppositeTest.happened && std
            ::abs(testOccluded.tnear - oppositeTest.tnear) < 0.1) {
            // calculate beta value of connection from camera to light
            cosTheta_ = std::max(0.0f, dotProduct(ints[i].inter.normal, current_w_
                ));
            if (i == 0) {
                pdf = ints[i].inter.material->pdf(ray.direction, current_w_,
                    ints[i].inter.normal);
            }
            else {
                pdf = ints[i].inter.material->pdf(w_s[i-1], current_w_, ints[i].
                    inter.normal);
            }

            albedo = ints[i].inter.obj->evalDiffuseColor(ints[i].inter.tcoords);
            f = ints[i].inter.material->eval(current_w_, ints[i].inter.normal,
                albedo);
            if (pdf < EPSILON) {
                continue;
            }

            Vector3f cameraBeta = f * cosTheta_ / std::max(pdf, EPSILON) /
                RussianRoulette;

            // Calculate geometry term
            Vector3f d = (ints[totalRaysFromCamera + j].inter.coords - ints[i].
                inter.coords) / 100;
            float distSquared = dotProduct(d, d);
            float dist = sqrt(distSquared);
            d = d / dist;
            float G = std::abs(dotProduct(ints[i].inter.normal, d)) * abs(
                dotProduct(ints[totalRaysFromCamera + j].inter.normal, -d)) /
                distSquared;

            if (distSquared < EPSILON) {
                continue; // colors can blow up if two points are too close
                    together
            }

            // Calculate color excluding MIS
            Vector3f color = ints[i].beta * ints[totalRaysFromCamera + j].beta *
                cameraBeta * G;

            // include MIS in final color
            hitColor += color / pSum * std::powf(camera_ps[i] * light_ps[j], 2);

```

```

    }
    }
}

return hitColor;
}

```

Listing 1: Bidirectional Path Tracing Implementation - `castRayBidirectional`

Scene Construction and Asset Generation

The custom chess pieces were initially modeled in Blender, triangulated, and exported as Wavefront .obj files. To recreate the reference framing, a virtual camera setup was used within the modeling software to extract the exact position offset vector for each individual piece. The surrounding environmental geometry was manually defined directly via object files. Once these assets were loaded into the scene, the final camera coordinates were fine-tuned to perfectly approximate the original perspective.

To accurately render the varying specular properties of the scene, a custom `DIFF_MIRROR` material was implemented. This material modifies the standard dielectric glass model by replacing the transmission (refraction) component with a standard Phong diffuse lobe, while retaining the specular reflection component. Crucially, the material dynamically computes the Fresnel reflectance based on the incident viewing angle and the specific Index of Refraction (IOR) assigned to the object. This algorithmic approach allows the reflectivity of a surface to scale physically without introducing unwanted transparency, enabling distinct, controllable specular profiles for the floor tiles, the lighter-colored pieces, and the darker, matte pieces.

Runtime Parameterization

The implementation of the dynamic scene configuration required refactoring the `Scene` and `Renderer` classes to decouple the camera state from the rendering logic. An `eye_pos` member variable was introduced to the `Scene` class definition within `Scene.hpp`. Consequently, the hardcoded camera coordinates previously embedded within `Renderer.cpp` (e.g., `Vector3f(278, 273, -800)`) were replaced with dynamic references to `scene.eye_pos`. This ensures that parameter values parsed at entry in `main.cpp` propagate correctly to the rendering loop.

Standard default values were preserved in the argument parser to maintain normal execution when run without custom flags. The executable now accepts the following parameterized argument structure:

```

./RayTracing <eye_x> <eye_y> <eye_z> <width> <height> <spp> <fov> \
    <king_x> <king_y> <king_z> <king_r> <king_g> <king_b> \
    <lx> <ly> <lz> <bg_r> <bg_g> <bg_b> <light_emission> \
    <dim_light_emission> <wall_r> <wall_g> <wall_b> \
    <smooth> <shadows_on> <bidirectional>

```

For example, generating a 256×256 render with 16 samples per pixel, an FOV of 40, and a translated camera Z-position can be executed identically across environments using:

```

./RayTracing 278 273 -1000 256 256 16 40

```

Because the argument parser cascades, omitting the remaining arguments safely leaves the scene materials and flags at their standard defaults. However, the CLI can also be fully expanded to override the entire scene state. For instance, generating that same render but utilizing low-poly models (for faster debugging) and disabling shadows would be executed by providing all arguments up to the boolean flags:

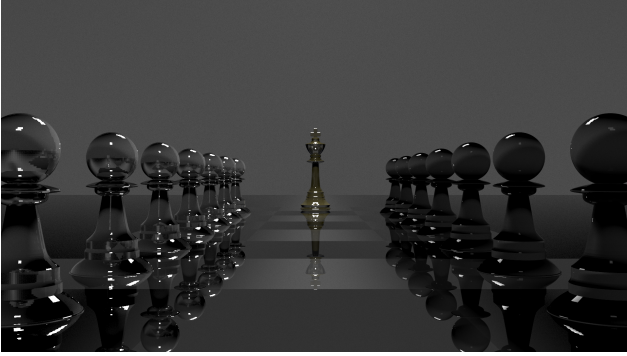
```

./RayTracing 278 273 -1000 256 256 16 40 \
0 0 7 0.27 0.25 \
0.12 0 0 8 0 0 \
0 400 100 0.12 0.12 0.12 0 0 1

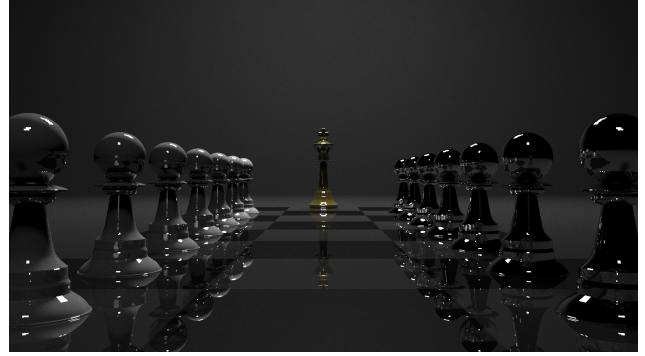
```

To systematically track the rendering experiments, all runtime configurations and their corresponding execution metrics were logged to a central tracking file, `outputs_log.csv`. Furthermore, all final image artifacts generated by these executions were automatically routed and saved to a dedicated `output_dir` directory, ensuring clean and organized management of the visual results.

Experiments and Results



(a) spp:64 ,rendered using standard path tracing



(b) spp:64 , rendered using BiDirectional path tracing

Discussion

References

- [1] Srinath Ravichandran. Dartmouth rendering competition, 2016.
- [2] Michal Vlnas. Bidirectional path tracing. In *"Proceedings of CESC 2018: The 22nd Central European Seminar on Computer Graphics"*, 2018.